UD1 — Desarrollo del software

Entornos de Desarrollo · 1.º DAW · 12 horas

Índice

1 Antes de empezar **1**
- 1.1 Qué vas a saber hacer al terminar esta unidad . . . 2
- 1.2 Por qué esta unidad importa (aunque parezca teórica) . . . 2

2 El software y el sistema informático **3**
- 2.1 Las tres capas . . . 3
- 2.2 Programa, aplicación, sistema . . . 3

3 Código fuente, objeto y ejecutable **4**
- 3.1 Definiciones . . . 4
- 3.2 La cadena completa en C . . . 4
- 3.3 El caso de Java: el bytecode . . . 5
- 3.4 El caso de JavaScript: interpretado con JIT . . . 5
- 3.5 Transpilación . . . 5

4 Clasificación de los lenguajes de programación **7**
- 4.1 Por nivel de abstracción . . . 7
- 4.2 Por propósito . . . 7
- 4.3 Por paradigma . . . 7
- 4.4 Por generación . . . 8

5 Las herramientas del proceso **9**
- 5.1 Bibliotecas: estáticas y dinámicas . . . 9

6 El ciclo de vida del software **11**
- 6.1 Las fases . . . 11

7 Modelos de ciclo de vida **13**
- 7.1 Cascada (*waterfall*) . . . 13
- 7.2 Prototipos . . . 13
- 7.3 Incremental . . . 13
- 7.4 Iterativo . . . 13
- 7.5 En espiral (Boehm) . . . 13
- 7.6 Metodologías ágiles . . . 13

8 Resumen de la unidad **16**

9 Autoevaluación **16**

1 Antes de empezar

1.1 Qué vas a saber hacer al terminar esta unidad

Esta unidad cubre el **RA1: identifica los elementos que intervienen en el desarrollo de software, analizando sus características**. Al acabarla tienes que ser capaz de:

- Explicar qué es un programa y en qué se diferencia de una aplicación.
- Distinguir código fuente, código objeto y código ejecutable, y decir cuál es cuál mirándolos.
- Clasificar un lenguaje de programación por nivel, propósito y paradigma.
- Describir cómo se obtiene el ejecutable en un lenguaje compilado, en uno interpretado y en uno que usa máquina virtual.
- Nombrar las herramientas que intervienen (compilador, enlazador, depurador, gestor de dependencias…) y decir qué hace cada una.
- Enumerar y explicar las fases del desarrollo de una aplicación, y comparar modelos de ciclo de vida.

1.2 Por qué esta unidad importa (aunque parezca teórica)

Vas a pasar dos años instalando cosas, ejecutando comandos y viendo errores. La diferencia entre alguien que resuelve un error en dos minutos y alguien que se atasca dos horas casi nunca es saber más sintaxis: es **entender qué está pasando por debajo**. Cuando `npm install` falla, o cuando el `.jar` no arranca, o cuando el navegador dice que no encuentra un módulo, lo que te salva es tener claro el camino que recorre tu código desde que lo escribes hasta que se ejecuta.

Eso es exactamente lo que hace esta unidad.

2 El software y el sistema informático

2.1 Las tres capas

Un sistema informático es hardware + software + personas. El software no flota: se apoya en capas, y cada capa solo habla con la de al lado.

Capa	Qué es	Ejemplos
Hardware	La máquina física	CPU, RAM, disco, tarjeta de red
Software de sistema	Hace utilizable el hardware	Sistema operativo, controladores, BIOS/UEFI
Software de programación	Sirve para construir software	Compiladores, IDE, depuradores
Software de aplicación	Resuelve un problema del usuario	Navegador, Photoshop, la web que vas a hacer

Tu código nunca habla con el disco duro. Le pide al sistema operativo que lo haga por él, mediante **llamadas al sistema**. Cuando en Node escribes:

```
const fs = require('node:fs');
fs.writeFileSync('tareas.json', datos);
```

lo que ocurre es: tu código → Node → libc → llamada al sistema `write()` → núcleo del SO → controlador de disco → hardware. Seis capas para guardar un fichero.

Idea clave

Todo lo que hace un programa que no sea calcular, lo hace pidiéndoselo al sistema operativo. Leer un fichero, abrir un socket, pintar una ventana, medir el tiempo. Por eso el mismo programa compilado para Windows no funciona en Linux: cambian las llamadas al sistema.

2.2 Programa, aplicación, sistema

Los tres términos se usan como sinónimos en la calle, pero en un examen no lo son:

- **Programa**: una secuencia de instrucciones que resuelve una tarea concreta. `ls`, `grep`, un script que renombra ficheros.
- **Aplicación informática**: un conjunto de programas, ficheros de datos, configuración y documentación que resuelve un problema completo de un usuario. Una aplicación web no es «un programa»: es servidor, base de datos, cliente, ficheros estáticos, scripts de despliegue.
- **Sistema de información**: la aplicación + las personas + los procedimientos de la organización que la usa.

Ojo

Pregunta típica de examen: *«¿Es WordPress un programa o una aplicación?»* Aplicación: son miles de ficheros PHP, una base de datos MySQL, plantillas, ficheros de configuración y documentación. Ningún fichero suelto hace nada por sí mismo.

3 Código fuente, objeto y ejecutable

Este es el apartado que más cae en las pruebas, y el que más rápido se entiende si lo ves en pantalla.

3.1 Definiciones

Código fuente: lo que escribe la persona. Texto plano, legible, escrito en un lenguaje de programación. Extensiones `.c`, `.java`, `.js`, `.py`. Se puede abrir con el Bloc de notas.

```c
// hola.c  — esto es código FUENTE
#include <stdio.h>
int main(void) {
    printf("Hola, mundo\n");
    return 0;
}
```

Código objeto: resultado intermedio de traducir el fuente a lenguaje máquina, pero **todavía no ejecutable**, porque le faltan las direcciones definitivas y el código de las bibliotecas que usa. Extensiones `.o` (Linux), `.obj` (Windows), `.class` (Java, aunque aquí es bytecode).

Código ejecutable: el resultado final, con todo resuelto, que el sistema operativo puede cargar en memoria y lanzar. Extensiones `.exe`, `.dll`, o sin extensión en Linux.

3.2 La cadena completa en C

```
hola.c   --[preprocesador]-->  hola.i   (fuente con los #include ya pegados)
hola.i   --[compilador]------>  hola.s   (ensamblador)
hola.s   --[ensamblador]----->  hola.o   (código OBJETO)
hola.o   --[enlazador]------->  hola     (código EJECUTABLE)
     + bibliotecas (libc...)
```

Cada flecha es un programa distinto. `gcc` los llama a todos por ti, pero puedes pararlo donde quieras:

```bash
gcc -E hola.c -o hola.i    # solo preprocesa
gcc -S hola.c              # genera hola.s (ensamblador)
gcc -c hola.c              # genera hola.o (objeto)
gcc hola.c -o hola         # todo: genera el ejecutable
```

Práctica

P1.1 — Ver los tres códigos con tus propios ojos (40 min)

1. Crea `hola.c` con el ejemplo de arriba.
2. Ejecuta los cuatro comandos anteriores, uno por uno.
3. Abre cada fichero resultante:
   - `hola.i` con VS Code. ¿Cuántas líneas tiene? ¿De dónde salen?
   - `hola.s` con VS Code. ¿Reconoces algo?
   - `hola.o` con VS Code. Verás basura: es binario. Ahora prueba `xxd hola.o | head -20`.
   - `hola` con `./hola`. Y luego `file hola` y `ls -l`.
4. Responde por escrito: ¿por qué `hola.o` ocupa menos que `hola`?
5. Repite `strings hola | head -30`. ¿Encuentras el texto «Hola, mundo»? ¿Y en `hola.o`?

Entrega: un documento con los cuatro ficheros abiertos (capturas) y las respuestas.

3.3 El caso de Java: el bytecode

Java rompe el esquema. `javac` no genera código máquina de tu procesador, sino **bytecode**: instrucciones para una máquina que no existe físicamente, la **JVM** (Java Virtual Machine).

```
Hola.java  --[javac]-->  Hola.class  --[java = JVM]-->  ejecución
                         (bytecode)
```

Ventaja: el mismo `.class` funciona en Windows, Linux y macOS, porque lo que cambia es la JVM, no tu fichero. Es el famoso *«write once, run anywhere»*. Inconveniente: necesitas tener la JVM instalada, y hay una capa extra de traducción en tiempo de ejecución.

```
javac Hola.java      # genera Hola.class
javap -c Hola        # ¡mira el bytecode desensamblado!
java Hola            # lo ejecuta la JVM
```

3.4 El caso de JavaScript: interpretado con JIT

JavaScript no se compila a fichero. El motor (V8 en Chrome y en Node) lee tu `.js`, lo analiza y lo ejecuta. Pero desde hace años los motores hacen **JIT** (*Just In Time*): mientras ejecutan, detectan qué funciones se usan mucho y las compilan a código máquina al vuelo.

```
app.js  --[V8: parser -> bytecode -> JIT]-->  ejecución
```

Por eso hoy no es cierto decir «JavaScript es lento porque es interpretado». Es más lento que C, sí, pero no por dos órdenes de magnitud como hace veinte años.

3.5 Transpilación

Un cuarto caso que vas a usar en 2.º: **transpilar** es traducir de un lenguaje de alto nivel a *otro lenguaje de alto nivel*. TypeScript → JavaScript, SASS → CSS, JSX → JavaScript. No hay código máquina por ningún lado.

Enfoque	Traduce a	Cuándo	Necesita en destino	Ejemplos
Compilado	Código máquina	Antes de ejecutar	Nada	C, C++, Rust, Go
Interpretado	Nada, se ejecuta directo	Durante	El intérprete	Python, PHP, Bash
Máquina virtual	Bytecode	Antes + durante	La VM	Java, C#, Kotlin
JIT	Máquina, al vuelo	Durante	El motor	JavaScript, Java moderno
Transpilado	Otro lenguaje alto	Antes	Lo que pida el destino	TypeScript, SASS

Práctica

P1.2 — El mismo algoritmo en tres mundos (2 h)
Implementa el mismo programa (pedir un número n y mostrar los n primeros números de Fibonacci) en **C**, **Java** y **JavaScript**.
Para cada uno rellena esta tabla:

	C	Java	JavaScript
Ficheros que escribes			
Comando para construir			

Ficheros intermedios generados
Comando para ejecutar
¿Qué hace falta en el ordenador de destino?
Tamaño total de lo entregado
Tiempo de `n = 40` (usa `time`)

Cuestión final: un cliente te pide un programa que funcione en Windows, Linux y Mac sin que él instale nada. ¿Cuál de los tres eliges y por qué? ¿Y si te pide que sea lo más rápido posible?

4 Clasificación de los lenguajes de programación

4.1 Por nivel de abstracción

- **Lenguaje máquina**: ceros y unos, el único que la CPU entiende de verdad. Nadie programa así desde los años 50.
- **Ensamblador**: mnemotécnicos (`MOV`, `ADD`, `JMP`) que corresponden 1 a 1 con instrucciones máquina. Dependiente del procesador. Se sigue usando en *drivers*, sistemas empotrados y optimizaciones extremas.
- **Alto nivel**: cercano al lenguaje humano e independiente del hardware. C, Java, Python, JavaScript. Aquí vivimos.

Ojo

Cuidado con un matiz clásico: **C es un lenguaje de alto nivel**, aunque a veces se le llame «de nivel medio» porque permite manipular memoria directamente. En un examen de FP, C es de alto nivel.

4.2 Por propósito

- **Propósito general**: sirven para casi todo. Java, C++, Python, C#.
- **Propósito específico**: hechos para un dominio. SQL (bases de datos), R (estadística), MATLAB (cálculo numérico), HTML/CSS (¡ojo, no son lenguajes de programación: son de marcas y de estilos, no tienen lógica de control!).

4.3 Por paradigma

Paradigma	Idea central	Lenguajes
Imperativo / estructurado	Secuencia de órdenes, con secuencia, selección e iteración	C, Pascal
Orientado a objetos	El programa es un conjunto de objetos que colaboran	Java, C#, C++
Funcional	El programa es composición de funciones sin estado mutable	Haskell, Elixir, Clojure
Lógico	Declaras hechos y reglas; el motor deduce	Prolog
Declarativo	Declaras *qué* quieres, no *cómo*	SQL, HTML
Multiparadigma	Varios a la vez	**JavaScript**, Python, Scala, Kotlin

JavaScript es el ejemplo perfecto de multiparadigma: puedes escribir código imperativo clásico, orientado a objetos con `class`, o funcional con `map/filter/reduce`.

```
// El mismo problema, tres paradigmas

// Imperativo
let suma = 0;
for (let i = 0; i < numeros.length; i++) {
  if (numeros[i] % 2 === 0) suma += numeros[i];
}

// Funcional
const suma = numeros.filter(n => n % 2 === 0)
                    .reduce((acc, n) => acc + n, 0);
```

```
// Orientado a objetos
class Coleccion {
  constructor(datos) { this.datos = datos; }
  sumaPares() { return this.datos.filter(n => n % 2 === 0).reduce((a, n) => a + n, 0); }
}
```

4.4 Por generación

Clasificación histórica que sigue apareciendo en los libros de texto:

1. **1.ª generación**: lenguaje máquina.
2. **2.ª generación**: ensamblador.
3. **3.ª generación**: alto nivel de propósito general (C, Java, JavaScript).
4. **4.ª generación**: muy próximos al dominio, generan código (SQL, generadores de informes).
5. **5.ª generación**: lógicos y orientados a IA (Prolog, LISP).

5 Las herramientas del proceso

Herramienta	Qué hace	Ejemplo
Editor	Escribir el código fuente	VS Code, Vim
Preprocesador	Prepara el fuente antes de compilar (macros, `#include`)	`cpp` en C
Compilador	Traduce fuente → objeto	`gcc`, `javac`, `tsc`
Intérprete	Lee y ejecuta el fuente línea a línea	`node`, `python`
Ensamblador	Traduce ensamblador → máquina	`as`, `nasm`
Enlazador (*linker*)	Une objetos y bibliotecas → ejecutable	`ld`
Cargador (*loader*)	Lleva el ejecutable a memoria y lo arranca	Parte del SO
Depurador	Ejecutar paso a paso, inspeccionar variables	`gdb`, depurador de VS Code
Gestor de dependencias	Descarga y versiona las bibliotecas de terceros	npm, Maven, pip
Automatizador de construcción	Orquesta todo el proceso	Make, Maven, Gradle, scripts de npm
Empaquetador (*bundler*)	Junta y optimiza el código para producción	Vite, webpack
Sistema de control de versiones	Guarda el historial del código	Git

5.1 Bibliotecas: estáticas y dinámicas

Cuando el enlazador une tu código objeto con una biblioteca, puede hacerlo de dos formas:

- **Enlazado estático**: el código de la biblioteca se **copia dentro** de tu ejecutable. El ejecutable es grande pero autosuficiente. Extensiones `.a` (Linux), `.lib` (Windows).
- **Enlazado dinámico**: el ejecutable solo guarda una **referencia**; la biblioteca se carga cuando el programa arranca. El ejecutable es pequeño, pero si la biblioteca falta o cambia de versión, el programa no arranca. Extensiones `.so` (Linux), `.dll` (Windows).

Idea clave

El temido «falta la DLL X» de Windows y el «error while loading shared libraries» de Linux son exactamente el mismo problema: **enlazado dinámico y la biblioteca no está**. En el mundo JavaScript su equivalente es `Cannot find module 'express'`: no has hecho `npm install`.

Práctica

P1.3 — Identifica la herramienta (30 min, en parejas)
Para cada mensaje de error, di qué herramienta lo ha producido y en qué fase del proceso:

1. `hola.c:4:5: error: expected ';' before 'return'`
2. `undefined reference to 'sqrt'`
3. `Error: Cannot find module 'express'`
4. `Exception in thread "main" java.lang.NullPointerException`
5. `bash: gcc: command not found`
6. `npm ERR! code ERESOLVE — unable to resolve dependency tree`
7. `SyntaxError: Unexpected token '}'` al abrir la web en Chrome
8. `error while loading shared libraries: libssl.so.1.1`

Para cada uno, indica además: ¿es un error de **compilación**, de **enlazado** o de **ejecución**?

6 El ciclo de vida del software

6.1 Las fases

Todo desarrollo, por pequeño que sea, pasa por estas fases. Que las hagas con papeleo formal o mentalmente en cinco minutos depende del proyecto; que las hagas, es inevitable.

6.1.1 1. Análisis de requisitos

Averiguar **qué** hay que construir. No cómo. Se habla con el cliente, se observa cómo trabaja hoy, se detectan contradicciones. Produce el **documento de requisitos** (ERS).

Dos tipos de requisitos:

- **Funcionales**: lo que el sistema debe hacer. *«El usuario podrá marcar una tarea como completada.»*
- **No funcionales**: cómo debe comportarse. *«La página debe cargar en menos de 2 segundos»*, *«debe funcionar en móvil»*, *«debe cumplir el RGPD»*.

Ojo

Es la fase más barata de arreglar y la más cara de equivocar. Un error de requisitos detectado en producción cuesta entre 50 y 200 veces más que detectado en análisis. Este dato lo popularizó Barry Boehm en los 80 y aunque las cifras exactas se discuten, el orden de magnitud se mantiene.

6.1.2 2. Diseño

Decidir **cómo** se va a construir. Arquitectura (¿cliente-servidor?, ¿en capas?), tecnologías, modelo de datos, división en módulos, interfaces entre ellos, diseño de la interfaz de usuario. Aquí es donde entran los **diagramas UML** que verás en UD5 y UD6.

6.1.3 3. Codificación

Escribir el código. Suele ser entre el 15 % y el 30 % del esfuerzo total del proyecto, aunque sea lo único que la gente imagina cuando piensa en «programar».

6.1.4 4. Pruebas

Comprobar que funciona y que hace lo que se pidió. Unitarias, de integración, de sistema, de aceptación. Es la UD3 entera.

6.1.5 5. Documentación

No es una fase que va al final: se hace **durante todo el proceso**. Tres destinatarios distintos:

- Documentación **de usuario**: manual, ayuda, tutoriales.
- Documentación **técnica**: para quien mantenga el código. JSDoc, diagramas, decisiones.
- Documentación **de proyecto**: requisitos, plan, actas, informes de pruebas.

6.1.6 6. Explotación (implantación)

Poner el software en producción: instalación, migración de datos, formación de usuarios, puesta en marcha.

6.1.7 7. Mantenimiento

Lo que pasa después, y donde se va entre el **60 % y el 80 % del coste total** del software a lo largo de su vida. Cuatro tipos:

Tipo	Qué es	Ejemplo
Correctivo	Arreglar errores	Un bug en el cálculo del IVA
Adaptativo	Ajustar a cambios del entorno	Migrar a PHP 8, nueva versión de Android
Perfectivo	Mejorar lo que ya funciona	Optimizar una consulta lenta, mejorar la UI
Preventivo	Evitar problemas futuros	Refactorizar, actualizar dependencias con CVE

Idea clave

Esta tabla explica por qué este módulo existe. Refactorizar (UD4), documentar (UD4), controlar versiones (UD4) y probar (UD3) son inversiones que se pagan en la fase de mantenimiento, es decir, en el 70 % del coste. Un proyecto sin pruebas ni documentación no es «más barato»: es más caro con la factura aplazada.

7 Modelos de ciclo de vida

Un modelo de ciclo de vida decide **en qué orden y cuántas veces** se recorren las fases.

7.1 Cascada (*waterfall*)

Las fases, una detrás de otra, sin volver atrás. No se empieza el diseño hasta que el análisis está aprobado y firmado.

`Análisis → Diseño → Codificación → Pruebas → Explotación → Mantenimiento`

- **A favor**: sencillo de planificar y presupuestar; mucha documentación; fácil de auditar.
- **En contra**: el cliente no ve nada hasta el final; si los requisitos cambian (y cambian), es carísimo; los errores de análisis se descubren tardísimo.
- **Cuándo tiene sentido**: requisitos muy estables y bien conocidos, entornos regulados (aviónica, sanidad, administración pública).

7.2 Prototipos

Se construye una maqueta rápida y desechable para que el cliente la vea y afine lo que quiere. Con los requisitos ya claros, se desarrolla en serio.

- **A favor**: resuelve el problema de «el cliente no sabe lo que quiere hasta que lo ve».
- **En contra**: riesgo real de que el prototipo acabe en producción («esto ya funciona, ¿no?»).

7.3 Incremental

El producto se entrega en trozos funcionales sucesivos. El incremento 1 es un sistema pequeño pero **completo y usable**; el 2 le añade funciones; etc.

- **A favor**: el cliente tiene valor desde pronto; el riesgo se reparte.
- **En contra**: exige una arquitectura pensada desde el principio para crecer.

7.4 Iterativo

Similar, pero cada vuelta **refina** el producto entero en lugar de añadirle trozos. En la práctica casi siempre se combinan: **iterativo e incremental**.

7.5 En espiral (Boehm)

Cada vuelta de la espiral tiene cuatro cuadrantes: determinar objetivos → **analizar riesgos** → desarrollar y verificar → planificar la siguiente vuelta. Lo distintivo es que el análisis de riesgos es explícito y dirige el proyecto.

- **Cuándo**: proyectos grandes, caros y con mucha incertidumbre técnica.

7.6 Metodologías ágiles

No son exactamente «un modelo de ciclo de vida» sino una filosofía, recogida en el **Manifiesto Ágil** (2001), con cuatro valores:

> Individuos e interacciones **sobre** procesos y herramientas. Software funcionando **sobre** documentación extensiva. Colaboración con el cliente **sobre** negociación contractual. Respuesta ante el cambio **sobre** seguir un plan.

Ojo

El error más repetido: leer «software funcionando sobre documentación extensiva» como «los ágiles no documentan». El manifiesto dice literalmente: *«aunque valoramos los elementos de la derecha, valoramos más los de la izquierda»*. Documentar sigue estando; documentar 300 páginas que nadie lee, no.

7.6.1 Scrum en 10 líneas

- El trabajo se organiza en **sprints** de 1 a 4 semanas.
- El **Product Owner** mantiene el *product backlog*: la lista priorizada de lo que hay que hacer, escrita como **historias de usuario** (*«Como usuario quiero… para…»*).
- En la **planificación de sprint** el equipo elige qué historias entran en el *sprint backlog*.
- Cada día hay una **daily** de 15 minutos: qué hice, qué haré, qué me bloquea.
- Al final del sprint hay una **revisión** (se enseña el incremento al cliente) y una **retrospectiva** (cómo mejorar la forma de trabajar).
- El **Scrum Master** elimina impedimentos y protege el proceso. No es un jefe.

7.6.2 Kanban en 5 líneas

Tablero con columnas (*Por hacer / En curso / En revisión / Hecho*), tarjetas que fluyen de izquierda a derecha y un **límite de trabajo en curso** por columna. Sin sprints ni roles obligatorios. Es lo que usa mucha gente en GitHub Projects o Trello.

Práctica

P1.4 — ¿Qué modelo usarías? (1 h, grupos de 3, con rúbrica)
Para cada encargo, elige un modelo de ciclo de vida y **justifícalo en 4-5 líneas**, citando al menos una ventaja y un riesgo:

1. Software de control de una máquina de radioterapia hospitalaria.
2. Una web para la peluquería de tu barrio: quiere «algo bonito donde se pueda pedir cita», y no sabe explicar mucho más.
3. El nuevo sistema de gestión académica de la Junta de Andalucía: 4 años, 30 personas, requisitos legales cambiantes.
4. Una *startup* que quiere lanzar una app de reparto y tiene 6 meses de financiación.
5. Migrar el ERP interno de una empresa de 200 empleados desde un sistema de 1998.

Entrega: un documento por grupo. Se valora la justificación, no la «respuesta correcta»: hay varias defendibles en casi todos los casos.

Práctica

P1.5 — El ciclo de vida de GestorTareas (1 h)
Vamos a arrancar el proyecto que nos acompañará todo el curso.

> **Enunciado del cliente**: «Necesito una herramienta para llevar mis tareas. Quiero poder apuntar una tarea con su título y su fecha límite, verlas todas, marcarlas como hechas y borrarlas. Me gustaría poder ponerles una etiqueta para agruparlas. De momento me vale con usarlo yo solo en mi ordenador, pero si funciona bien quizá lo quiera en web.»

En grupo, redactad un documento con:

1. **Requisitos funcionales** numerados (RF1, RF2…). Mínimo 6.
2. **Requisitos no funcionales** (RNF1, RNF2…). Mínimo 3.
3. **Requisitos ambiguos** que habría que preguntar al cliente. Mínimo 3. (Pista: ¿qué pasa si se borra una tarea por error? ¿la fecha límite es obligatoria?)
4. Una tabla con las **7 fases** y, en cada una, qué haríais concretamente **en este proyecto**.

5. Qué **modelo de ciclo de vida** proponéis y por qué.

Este documento es la entrada de todo lo que haremos en UD2, UD5 y UD6. Guardadlo bien.

8 Resumen de la unidad

Concepto	En una línea
Programa vs aplicación	Un programa resuelve una tarea; una aplicación es el conjunto completo
Código fuente	Texto que escribe la persona
Código objeto	Traducido a máquina pero sin enlazar
Código ejecutable	Listo para que el SO lo cargue y lo lance
Compilado	Se traduce entero antes de ejecutar (C)
Interpretado	Se lee y ejecuta sobre la marcha (PHP, Bash)
Máquina virtual	Se compila a bytecode y lo ejecuta una VM (Java)
JIT	Se compila a máquina durante la ejecución (JavaScript/V8)
Transpilado	Se traduce a otro lenguaje de alto nivel (TypeScript)
Enlazado estático	La biblioteca se copia dentro del ejecutable
Enlazado dinámico	La biblioteca se carga al arrancar (`.dll`, `.so`)
Fases del ciclo de vida	Análisis, diseño, codificación, pruebas, documentación, explotación, mantenimiento
Mantenimiento	60-80 % del coste total; correctivo, adaptativo, perfectivo, preventivo
Cascada	Fases en orden, sin vuelta atrás
Incremental	Entregas sucesivas completas y usables
Espiral	Iterativo con análisis de riesgos explícito
Ágil	Manifiesto 2001; Scrum (sprints, backlog, roles) y Kanban (flujo, WIP)

9 Autoevaluación

1. ¿Puede existir código objeto sin código fuente? ¿Y código ejecutable sin código objeto?
2. Tienes un `.class` de Java. ¿Funcionará en un Mac con procesador ARM? ¿Y un `.exe` compilado en Windows x86?
3. ¿Por qué un ejecutable enlazado estáticamente ocupa más pero es más «portable»?
4. Un compañero dice: «HTML es un lenguaje de programación de alto nivel». Corrígelo con argumentos.
5. Ordena de más barato a más caro corregir un error detectado en: producción, pruebas, análisis, codificación.
6. Tu jefe dice: «vamos a hacerlo ágil, así no perdemos tiempo documentando». ¿Qué le contestas?
7. Clasifica el mantenimiento: (a) actualizas una dependencia porque tiene una vulnerabilidad;
   (b) el cliente pide un botón nuevo; (c) la app deja de funcionar en iOS 19; (d) arreglas un cálculo mal hecho.
8. ¿Qué diferencia hay entre un enlazador y un cargador?